# Szakdolgozat

Miskolci Egyetem

# JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata

**Készítette:**

Sándor János

Programtervező informatikus

**Témavezető:**

Dr. Agárdi Anita

Miskolc, 2026

**Miskolci Egyetem**
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

# Szakdolgozat Feladat

Sándor János (U03GDK) programtervező informatikus jelölt részére.

**A szakdolgozat tárgyköre:** webes alkalmazás fejlesztése

**A szakdolgozat címe:** JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata

**A feladat részletezése:**

*Az alkalmazás fejlesztése JavaScript alapú eszközök kerülnek felhasználásra, mint a React, React Native és Express.js. A szakdolgozat célja, hogy egy (főként szoftverfejlesztési) projektek menedzselésére alkalmas webalkalmazás fejlesztése kapcsán vizsgálja a JavaScript alapú webalkalmazások karbantarthatóságát. A dolgozat kitér a projekt kezelés általános működésére, típusaira. Összehasonlítja az aktuálisan elérhető alternatívákat. Specifikálja a készülő rendszer működését. Az alkalmazás készítése kapcsán kiemelt szerepet kap az alkalmazás forráskódjának karbantarthatósága, figyelembe véve a forráskód szerkezetét és a szoftverfejlesztési módszereket.*

*Az alkalmazás fejlesztéséhez JavaScript alapú eszközök kerülnek felhasználásra, mint a React, React Native és Express.js.*

**Témavezető:** Dr. Agárdi Anita egyetemi adjunktus

**A feladat kiadásának ideje:**

. . . . . .
szakfelelős

# Eredetiségi Nyilatkozat

Alulírott **Sándor János**; Neptun-kód: `U03GDK` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *JavaScript alapú webalkalmazások karbantarthatóságának vizsgálata* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ..... .év ..... .hó ..... .nap

.....

Hallgató

1.

A szakdolgozat feladat módosítása szükséges (módosítás külön lapon)

nem szükséges

..... .....

dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás): konzulens (dátum, aláírás):

..... .....

..... .....

..... .....

3. A szakdolgozat beadható:

..... .....

dátum témavezető(k)

4. A szakdolgozat ..... szövegoldalt

..... program protokollt (listát, felhasználói leírást)

..... elektronikus adathordozót (részletezve)

.....

..... egyéb mellékletet (részletezve)

.....

tartalmaz.

..... .....

dátum témavezető(k)

5.

A szakdolgozat bírálatra bocsátható

nem bocsátható

A bíráló neve: .....

..... .....

dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata: .....

a bíráló javaslata: .....

a szakdolgozat végleges eredménye: .....

Miskolc, ..... .....

a Záróvizsga Bizottság Elnöke

# Tartalomjegyzék

# 1. fejezet

# Bevezetés

A mindennapi életben végzett tevékenységeink szervezése és nyomon követése az utóbbi években alapvetően megváltozott. A digitalizáció előretörése azt eredményezte, hogy a feladatkezelés már korántsem csupán papíralapú jegyzetfüzetekben, falinaptárakban vagy post-it cédulákon történik. A számítógépek, okostelefonok és felhőalapú szolgáltatások korszakában számos olyan alkalmazás vált széles körben elérhetővé, amelynek célja a feladatok rendszerezése, a határidők figyelése és a személyes hatékonyság növelése. Mégis, a felhasználók jelentős része továbbra is azt tapasztalja, hogy a feladatkezelés nehezen átlátható, túl bonyolult vagy egyszerűen nem illeszkedik a saját gondolkodási folyamataikhoz. Ez a paradoxon – miszerint a rengeteg eszköz ellenére a feladatkezelés nem feltétlenül válik könnyebbé – hívta életre azt az igényt, hogy újragondoljuk, mit is várunk egy modern feladatkezelő rendszertől.

A probléma gyökere abban rejlik, hogy az emberek munkastílusa, gondolkodása és prioritáskezelése rendkívül különböző. Egyesek listákban szeretnek gondolkodni, mások vizuális naptárakban, vannak, akik az egyszerre egy feladatra fókuszálást részesítik előnyben, és olyanok is, akiknek az átlátható projektszerkezet a legfontosabb. A ma elérhető eszközök döntő többsége azonban egy előre meghatározott feladatkezelési filozófiát próbál a felhasználóra erőltetni: vagy listaalapú működést biztosít, vagy kártyaorientált felületet, vagy szigorúan GTD kompatibilis struktúrát, esetleg valamilyen saját módszertant. Ezek az eszközök hatékonyak lehetnek azok számára, akiknek a gondolkodása illeszkedik az adott működésmódhoz, azonban sok felhasználó egyszerűen nem találja meg bennük azt a rugalmasságot és személyre szabhatóságot, amely hosszú távon is fenntartható használatot eredményezne. Emiatt gyakori jelenség, hogy valaki nagy lelkesedéssel kezdi használni az új rendszert, majd néhány hét vagy hónap után teljesen elengedi, mert túl sok munka fenntartani, vagy éppen túl kevés értéket ad a mindennapokhoz.

A feladatkezelők másik általános problémája, hogy sokszor csak felületi változatosságot kínálnak, tartalmi rugalmasságot viszont alig. A feladatokat gyakran ugyanúgy kell megadni: név, határidő, címkék és esetleg valamilyen kategória. Bár ezek alapvető elemek, az emberi gondolkodás ennél jóval gazdagabb. A feladatokhoz kapcsolódhatnak összefüggések, prioritások, előfeltételek, kontextusok, amelyek figyelmen kívül hagyása a rendszer használatát korlátozottá teszi. Emellett az is problémát jelent, hogy a legtöbb eszköz a feladatokat felhasználói szinten egyetlen nagy masszaként kezeli, miközben a felhasználó élete nem egyetlen síkban zajlik. Másképp gondolkodunk a munkáról, a magánéletről, a hosszú távú célokról vagy akár egy hétköznapi bevásárlólistáról.

Ezekre a kihívásokra válaszul született meg az a felismerés, hogy szükség lenne egy

olyan rendszerre, amely nem előre definiált struktúrákat kényszerít a felhasználóra, hanem képes alkalmazkodni hozzá. Egy olyan rendszerre, amely nem csak eszköz, hanem nyelvezet is: amelyben a felhasználó saját gondolkodásához igazíthatja a feladatok és információk szervezésének módját. Ezt az igényt igyekszik kielégíteni az Acxor, amely egy olyan, rugalmas és földre eső feladatkezelési modellt kínál, amely a feladatokat több dimenzió mentén képes értelmezni. Nem egyszerűen listák vagy projektek halmazát kezeli, hanem egymással összefüggő, kontextusba ágyazott elemeket, amelyek tetszőleges módon kombinálhatók.

A rendszer újszerűsége abban rejlik, hogy a feladatokat nem kizárólag hagyományos értelemben vett teendőként kezeli, hanem olyan absztrakciókként, amelyek kapcsolatokat, célokat, állapotokat és prioritásokat is hordozhatnak. A modell lehetővé teszi, hogy a felhasználó egyetlen nézőpont-váltással egészen más jellegű információt kapjon ugyanarról az adathalmazról: láthatja időbeli sorrendben, kontextusok szerint, projektekre bontva vagy akár egyedi szűrők alapján. Az Acxor alapelve az, hogy a rendszernek kell alkalmazkodnia a felhasználóhoz, nem pedig fordítva. Ezzel a megközelítéssel olyan rugalmasság érhető el, amely ritkán található meg a jelenlegi feladatkezelő rendszerekben.

A dolgozat célja ennek a koncepciónak a részletes bemutatása, valamint egy olyan működési modell kidolgozása, amely mind a technikai megvalósítást, mind a felhasználói élményt tekintve életképes és bővíthető alapot biztosít egy későbbi fejlesztéshez. A bevezető fejezet ezért elsősorban azt kívánja megalapozni, hogy miért érdemes ezt a témát kutatni, és milyen gyakorlati igény hívta életre az ötletet. A következő fejezetek majd részletesen bemutatják azokat az elméleti modelleket, amelyekre a koncepció épül, valamint azokat az eszközöket és technikákat, amelyek segítségével egy modern, rugalmas feladatkezelő rendszer megvalósítható. A jelen fejezet feladata inkább az, hogy érzékeltesse a probléma súlyát, és megmutassa, hogy a fejlesztett rendszer milyen társadalmi és technológiai környezetben nyer értelmet.

Összességében tehát a dolgozat egy olyan problémára koncentrál, amely egyszerre technikai, pszichológiai és felhasználói élménybeli kihívás: hogyan lehet olyan feladatkezelő rendszert alkotni, amely valóban segít, és nem újabb terhet jelent? Hogyan lehet egy rendszert úgy felépíteni, hogy az alkalmazkodjon a felhasználó gondolkodásmódjához, és ne kényszerítse bele egy előre meghatározott struktúrába? És hogyan lehet mindezt úgy megtenni, hogy a rendszer mégis átlátható, bővíthető és hosszú távon fenntartható maradjon? E kérdések megválaszolása nem csupán érdekes, hanem egyben aktuális és összetett feladat is, hiszen a digitális produktivitási eszközök környezete gyorsan változik, és a felhasználói igények is folyamatosan fejlődnek.

A dolgozat végső célja olyan modell bemutatása, amely új alapokra helyezi a feladatkezelésről való gondolkodást, és megmutatja, hogy a feladatkezelő rendszerek nem feltétlenül kell, hogy merev és előre meghatározott sémák szerint működjenek. Ezzel nem csupán egy új eszköz koncepcióját vázolja fel, hanem hozzájárul a feladatkezelési rendszerek jövőjéről folytatott szakmai párbeszédhez is.

# 2. fejezet

# Koncepció

## 2.1. A fejezet célja

Ebben a fejezetben a dolgozat témáját körülhatároljuk, bemutatjuk a releváns projektmenedzsment rendszereket, azok főbb funkcióit, előnyeit és hátrányait, valamint azt, hogyan kapcsolódnak az Acxor fejlesztéséhez. A cél nem a saját eredmények részletezése, hanem egy átfogó, irodalmi és technológiai alap bemutatása, amelyre a dolgozat további részei építenek. A hivatkozások jelentős része ebben a fejezetben kerül alkalmazásra, például [**?**].

## 2.2. Tartalom és felépítés

A fejezet tartalma négy fő területre koncentrál: irodalomkutatás, technológia, piackutatás és követelmény specifikáció. Az irodalomkutatás során áttekintjük a legfontosabb projektmenedzsment rendszereket, a technológiai részben bemutatjuk a rendszerek főbb funkcióit, a piackutatás során összehasonlítjuk a meglévő megoldásokat, míg a követelmény specifikációban az Acxor fejlesztés céljait és alapelveit foglaljuk össze.

## 2.3. Projektmenedzsment rendszerek áttekintése

### 2.3.1. JIRA

A JIRA egy komplex, vállalati szintű projektmenedzsment és issue tracking rendszer. Kiemelt jellemzői [**?**]:

- Issue tracking, Agile metodikák (Scrum / Kanban) támogatása
- Workflow testreszabás
- Jogosultságkezelés
- Riportok és dashboardok
- Széleskörű integrációk (Confluence, Bitbucket, GitHub, Slack, Teams)

A JIRA rugalmas és szinte bármi testreszabható, ami előnyt jelent nagyvállalati környezetben, ugyanakkor komplexitása miatt tanulási görbéje meredek. A rendszer

hierarchiája Epic–Story–Task–Subtask struktúrát követ, ami előnyös a nagy csapatok számára. Automatizmusai trigger–condition–action logikán alapulnak, amely lehetővé teszi például automatikus állapotváltást, assignee hozzárendelést, vagy e-mail értesítések küldését.

Az Acxor fejlesztésében a JIRA inspirációként szolgált a funkcionalitás és a workflow kezelés terén, de egyszerűsített felhasználói felületet és átláthatóbb jogosultsági modellt alkalmaz.

### 2.3.2. Trello

A Trello egy intuitív, kártya-alapú projektmenedzsment rendszer, amely kisebb csapatoknak vagy egyszerűbb folyamatokhoz ideális. Főbb jellemzői [**?**]:

- Board → List → Card → Checklist vizuális hierarchia
- Drag & drop alapú kezelés
- Minimális konfiguráció és real-time szinkron
- Egyszerű automatizmusok (Butler)

A Trello előnye a gyors betanulás és a vizuális, átlátható felület, míg hátránya a korlátozott testreszabhatóság és a nagyobb projektekhez kevésbé megfelelő komplexitás. Az Acxor esetében a Trello inspirálta az intuitív, vizuális task lifecycle és board kezelést, amely egyszerre egyszerű és professzionális.

### 2.3.3. Asana

Az Asana egy feladat- és projektmenedzsment platform, amely a csapatok együttműködését támogatja. Fő funkciói [**?**]:

- Task és Subtask kezelés
- Projektek és Timeline vizualizáció
- Határidők és felelősök hozzárendelése
- Dashboard és riporting
- Integráció külső szolgáltatásokkal (Slack, Teams, GitHub, Google Workspace)

Előnye a könnyen átlátható felület és a gyors onboarding, hátránya a korlátozott workflow testreszabás és a részletes riporting hiánya. Az Acxor tervezésénél az Asana inspirációt jelentett a fókuszált task lifecycle és minimalista felület kialakításában.

### 2.3.4. Redmine

A Redmine nyílt forráskódú rendszer, amely elsősorban testreszabhatóságáról és bővíthetőségéről ismert. Főbb jellemzői [?]:

- Issue tracking és projekt hierarchia (főprojekt → alprojekt)
- Testreszabható workflow és permission rendszer
- Gantt és Calendar vizualizáció
- Plugin és API alapú integráció

Előnye a nagy projektek kezelésére való alkalmasság és a nyílt forráskódú bővíthetőség, míg hátránya a kevésbé modern UI és a tanulási görbe. Az Acxorban a Redmine inspirációt jelentett a backend struktúrában és az issue hierarchia egyszerűsítésében, modern, letisztult felülettel.

### 2.3.5. Projektmenedzsment rendszerek összehasonlítása

Az Acxor a közepes komplexitású fejlesztői csapatok számára készült, kiemelten figyelve a vizualitásra, egyszerűségre és motivációs rendszerekre. Összehasonlításképpen bemutatom a Redmine, Asana, Trello és Jira főbb jellemzőit:

| **Funkció / Eszköz** | **Acxor** | **Trello** | **Jira** |
|---|---|---|---|
| Webes felület | Igen, modern vizuális UI | Igen, webes, vizuális | Igen, webes, komplex |
| Kanban board | Igen, beépített, drag&drop | Igen, beépített | Részben, plugin / board szükséges |
| Sprint / Iteration | Részben, egyszerű | Nem | Igen, teljes körű |
| Gamifikáció | Igen, XP, szint, badge | Nem | Nem |
| Task / Subtask | Igen, Epic → Story Task → Subtask | Igen, kártya / checklist | Igen, Epic → Story Task → Subtask |
| Chat / Log | Igen, globális bot | Részben, kommentek | Részben, jegy alapú |
| Workflow testreszabás | Részben, egyszerű | Részben, limitált | Igen, részletes, szabály alapú |
| Permission rendszer | Részben, egyszerű | Részben, egyszerű | Igen, mély, szerepkör alapú |
| Integrációk | Igen, alap, kulcs szolgáltatások | Részben, Power-Ups | Igen, széles (Confluence, GitHub, Slack...) |

2.1. táblázat. Acxor összehasonlítása a Trello-val és a Jira-val

## 2.4. Összegzés

A fenti rendszerek áttekintése rávilágít arra, hogy a projektmenedzsment eszközök széles skálán helyezkednek el komplexitás, testreszabhatóság és felhasználói élmény szempontjából. Az Acxor célja a különböző rendszerek előnyeinek ötvözése: a JIRA funkcionalitásának, a Trello vizuális egyszerűségének, az Asana átlátható task lifecycle-jának, valamint a Redmine struktúrájának kombinálása egy intuitív, könnyen használható, ugyanakkor professzionális projektmenedzsment platformban.

Ez a koncepciói alap biztosítja, hogy a dolgozat későbbi fejezeteiben bemutatott fejlesztések jól meghatározott, átgondolt irányelv szerint történjenek, és az olvasó számára érthető, követhető legyen a választott megközelítés és technológiai háttér.

| **Funkció / Eszköz** | **Acxor** | **Redmine** | **Asana** |
|---|---|---|---|
| Webes felület | Igen, modern vizuális UI | Igen, webes, részletes | Igen, webes, intuitív |
| Kanban board | Igen, beépített | Részben, plugin szükséges | Igen, board / list nézet |
| Sprint / Iteration | Részben, egyszerű | Igen, teljes körű | Részben, idővonal / Timeline |
| Gamifikáció | Igen, XP, szint, badge | Nem | Nem |
| Task / Subtask | Igen, egyszerű hierarchia | Igen, issue / subtask | Igen, task / subtask |
| Chat / Log | Igen, globális bot | Részben, jegy alapú | Részben, kommentek |
| Workflow testreszabás | Részben, egyszerű | Igen, részletes | Részben, egyszerű |
| Permission rendszer | Részben, egyszerű | Igen, roles | Részben, alap |
| Integrációk | Igen, kulcs szolgáltatások | Részben, pluginok, SCM | Igen, Slack, GitHub, Google Workspace |

2.2. táblázat. Acxor összehasonlítása a Redmine-nel és az Asana-val

# 3. fejezet

# Tervezés

Itt kezdődik a dolgozat lényegi része, úgy értve, hogy a saját munka bemutatása. Jellemzően ebben szerepelni szoktak blokkdiagramok, a program struktúrájával foglalkozó leírások. Ehhez célszerű UML ábrákat (például osztály- és szekvenciadiagramokat) használni.

Amennyiben a dolgozat inkább kutatás jellegű, úgy itt lehet konkretizálni a kutatási módszertant, a kutatás tervezett lépéseit, az indoklást, hogy mit, miért és miért pont úgy érdemes csinálni, ahogyan az a későbbiekben majd részletezésre kerül.

Ebben a fejezetben az implementáció nem kell, hogy túl nagy szerepet kapjon. Ez még csak a tervezési fázis. (Nyilván ha olyan a téma, hogy magának az implementációnak a módjával foglalkozik, adott formális nyelvet mutat be, úgy a kódpéldákat már innen sem lehet kihagyni.)

## 3.1. Táblázatok

Táblázatokhoz a `table` környezetet ajánlott használni. Erre egy minta a **??**. táblázat. A hivatkozáshoz az egyedi `label` értéke konvenció szerint `tab:` prefixszel kezdődik.

3.1. táblázat. Minta táblázat. A táblázat felirata a táblázat felett kell legyen!

| a | b | c |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |

## 3.2. Ábrák

Ábrákat a `figure` környezettel lehet használni. A használatára egy példa a **??**. ábrán látható. Az `includegraphics` parancsba Az ábrák felirata az ábra alatt kell legyen. Az ábrák hivatkozásához használt nevet konvenció szerint `fig:`-el célszerű kezdeni.

## 3.3. További környezetek

A matematikai témájú dolgozatokban szükség lehet tételek és bizonyításaik megadására. Ehhez szintén vannak készen elérhető környezetek.

3.1. ábra. A Miskolci Egyetem címere.

**3.1. definíció.** Ez egy definíció

**3.2. lemma.** *Ez egy lemma*

**3.3. tétel.** *Ez egy tétel*

**Bizonyítás.** Ez egy bizonyítás □

**3.4. következmény.** *Ez egy tétel*

**3.5. megjegyzés.** Ez egy megjegyzés

**3.6. példa.** Ez egy példa

# 4. fejezet

# Megvalósítás

Ez a fejezet mutatja be a megvalósítás lépéseit. Itt lehet az esetlegesen előforduló technikai nehézségeket említeni. Be lehet már mutatni a program elkészült részeit.

Meg lehet mutatni az elkészített programkód érdekesebb részeit. (Az érdekesebb részek bemutatására kellene szorítkozni. Többségében a szöveges leírásnak kellene benne lennie. Abból lehet kiindulni, hogy a forráskód a dolgozathoz elérhető, azt nem kell magába a dolgozatba bemásolni, elegendő csak behivatkozni.)

A dolgozatban szereplő forráskódrészletekhez külön vannak programnyelvenként stílusok. Python esetében például így néz ki egy formázott kódrészlet.

```python
import sys

if __name__ == '__main__':
    pass
```

A stílusfájlok a `styles` jegyzékben találhatók. A stílusok között szerepel még C++, Java és Rust stílusfájl. Ezek használatához a `dolgozat.tex` fájl elején `usepackage` paranccsal hozzá kell adni a stílust, majd a stílusfájl nevével megegyező környezetet lehet használni. További példaként C++ forráskód esetében ez így szerepel.

```cpp
#include <iostream>

class Sample : public Object
{
    // An empty class definition
}
```

Stílusfájlokból elegendő csak annyit meghagyni, amennyire a dolgozatban szükség van. Más, C szintaktikájú nyelvekhez (mint például a JavaScript és C#) a Java vagy C++ stílusfájlok átszerkesztésére van szükség. (Elegendő lehet csak a fájlnevet átírni, és a fájlban a környezet nevét.)

Nyers adatok, parancssori kimenetek megjelenítéséhez a `verbatim` környezetet lehet használni.

```
$ some commands with arguments
1 2 3 4 5
$ _
```

A kutatás jellegű témáknál ez a fejezet gyakorlatilag kimaradhat. Helyette inkább a fő vizsgálati módszerek, kutatási irányok kaphatnak külön-külön fejezeteket.

# 5. fejezet

# Tesztelés

A fejezetben be kell mutatni, hogy az elkészült alkalmazás hogyan használható. (Az, hogy hogyan kell, hogy működjön, és hogy hogy lett elkészítve, az előző fejezetekben már megtörtént.)

Jellemzően az alábbi dolgok kerülhetnek ide.

- Tesztfuttatások. Le lehet írni a futási időket, memória és tárigényt.
- Felhasználói kézikönyv jellegű leírás. Kifejezetten a végfelhasználó szempontjából lehet azt bemutatni, hogy mit hogy lehet majd használni.
- Kutatás kapcsán ide főként táblázatok, görbék és egyéb részletes összesítések kerülhetnek.

# 6. fejezet

# Összefoglalás

Hasonló szerepe van, mint a bevezetésnek. Itt már múltidőben lehet beszélni. A szerző saját meglátása szerint kell összegezni és értékelni a dolgozat fontosabb eredményeit. Meg lehet benne említeni, hogy mi az ami jobban, mi az ami kevésbé jobban sikerült a tervezettnél. El lehet benne mondani, hogy milyen további tervek, fejlesztési lehetőségek vannak még a témával kapcsolatban.

# CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy MarkDown fájlként kimentve).